



MONASH
University

IB Australia
Technical Documentation Report

By Team 17

V2.0
18/09/2023

Table of Contents

Table of Contents	2
Code repository: Full Clone	3
SQL dump	3
High level system architecture diagram with servers/databases	3
Database ERD Model	4
Codebase structure	4
Main folders	4
src	5
Development Environment	5
IDEs	5
Instruction of running website locally	5
Required Skills:	8
External APIs	8
Authorisation:	8
All MYOB header requirements:	9
Load customer API:	9
Load customer API:	12
Create Order API:	15
Deployment documentation	16
Design Specifications	17
Typography details	17
Brand Palette	17
UI/UX components	19
Icons	19
Logo designs	19
Images	20

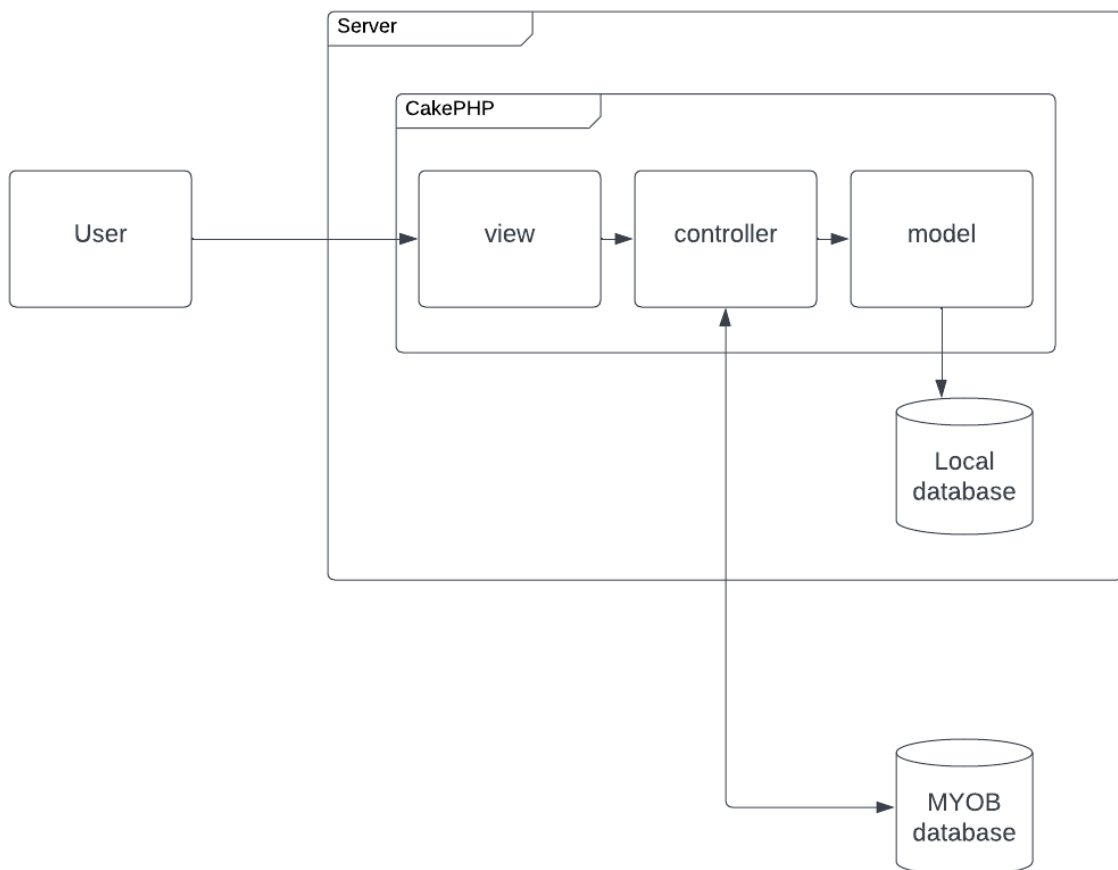
Code repository: Full Clone

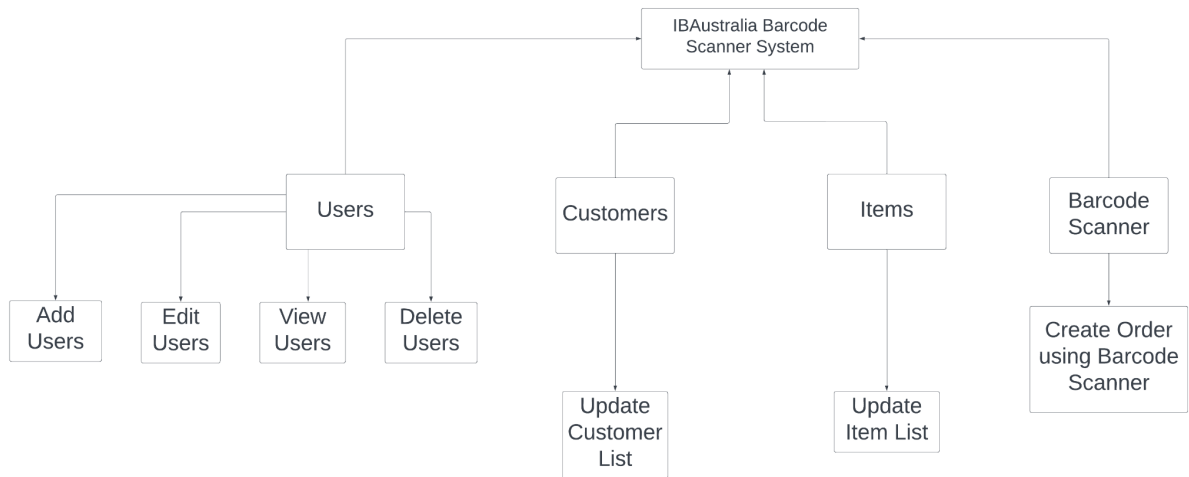
A zip is stored alongside this document

SQL dump

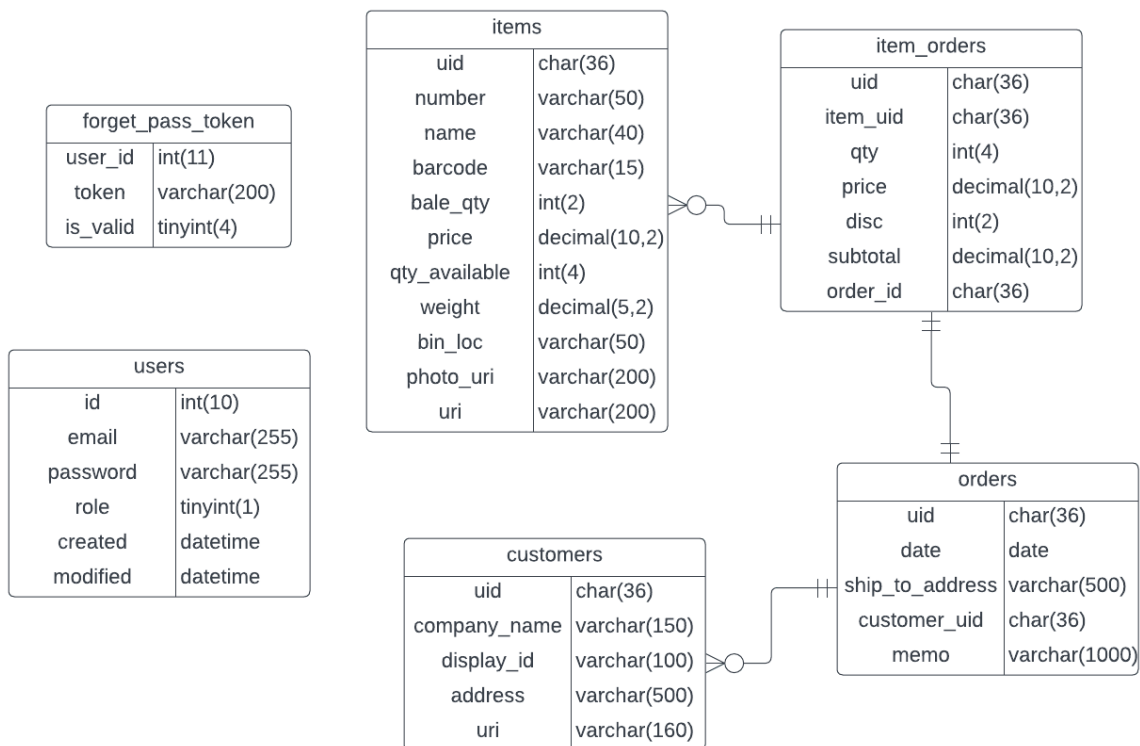
A SQL file is stored alongside this document

High level system architecture diagram with servers/databases





Database ERD Model



Codebase structure

Main folders

- *bin*: Location of running any cake commands from the command line
- *config*: Stores all the configurations for your code to run
- *plugins*: Storage of plugins. Eg. authentication plugin
- *logs*: Storage of latest error and debug logs

- *src*: Storage of source files for the backend code
- *resources*: unused - would usually store various resource files
- *templates*: Specifies the html of the pages that you see in the UI, split into folders of what *controller* they're connected to, then further into each page which is connected to one function in the controller
- *tests*: Test cases for controllers, tables and templates - we haven't edited this
- *tmp*: Temporary cached data. The specific data depends on your configurations - we haven't edited this
- *vendor*: Application dependencies that are installed by composer. Even if you manually change this, this will be updated the next time you run composer
- *webroot*: public root of the application. All files you want to be publicly reachable are here. These are things like your css, js files and any images you wish to display

src

- *src.console*: Has the installation script used by Composer.
- *src.Controller*: The backend logic and connection to other APIs and databases
- *src.Model.Behaviour*: unused
- *src.Model.Entity*: A Plain Old PHP Object (POPO) which defines the classes we use and which attributes they have
- *src.Model.Table*: Helper functions for connections to databases like getting or saving to a database
- *src.View*: Presentational classes like views, cells and helpers will be put here, but we haven't used this

Development Environment

IDEs

JetBrains-Phpstorm:

PhpStorm is a development tool for PHP and Web projects. It's a perfect PHP IDE for working with Laravel, Symfony, Drupal, WordPress, CakePhp, and other frameworks.

<https://www.jetbrains.com/phpstorm/features/>

Microsoft Corporation-Visual Studio Code:

Visual Studio Code is a code editor redefined and optimised for building and debugging modern web and cloud applications.

<https://code.visualstudio.com/docs>

Instruction of running website locally

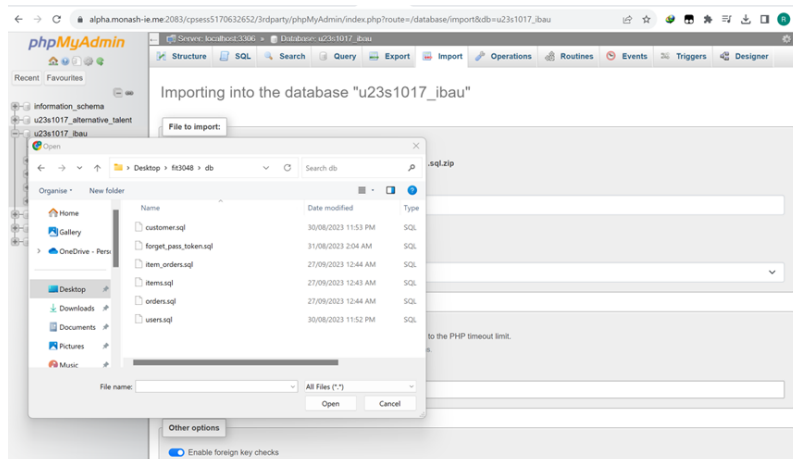
If you had experience of running cakephp project locally in the past, please skip to step4

1. Install Cakephp into your computer. <https://book.cakephp.org/4/en/installation.html>

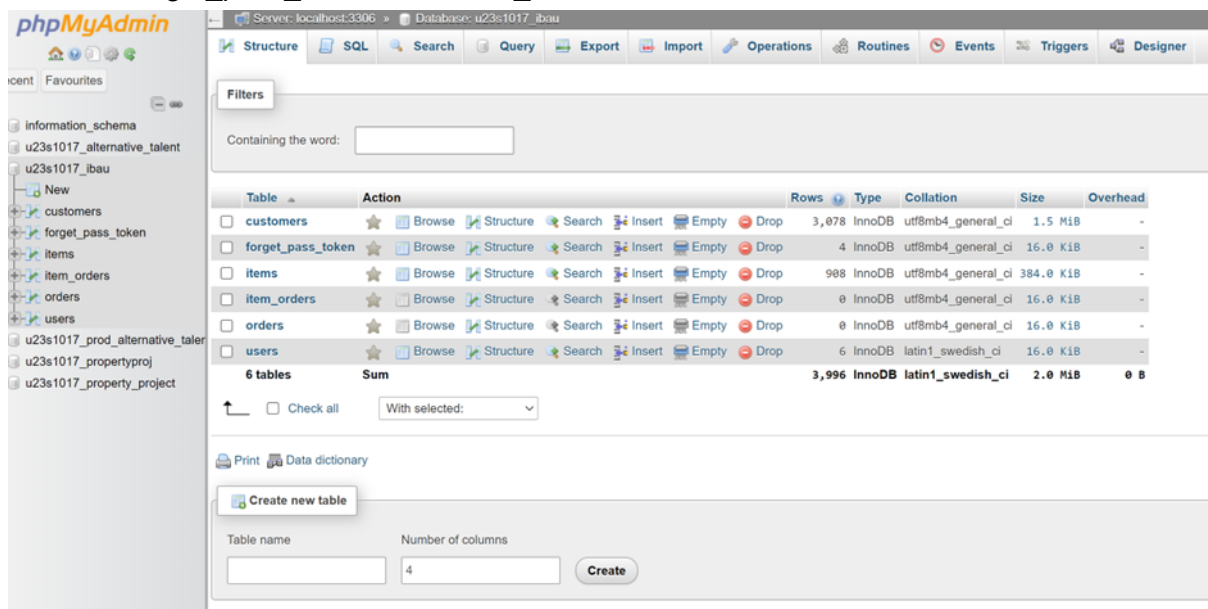
Then go to

<https://www.techpowerup.com/download/visual-c-redisistributable-runtime-package-all-in-one/>

- and download the zip file. Then extract it to a folder and you should see a file named `install_all`, click on it to install all the libraries that cakephp need
2. Install XAMPP. <https://www.apachefriends.org/> After installed, then open the XAMPP Control Panel in your computer. Tick box the Apache and MySQL modules to install them, then press 'start' actions to start these modules
3. Download and run Composer-Setup.exe on <https://getcomposer.org/download/> to install the composer
4. Open browser and goto <http://localhost/phpmyadmin/> Create a new database then import the sql files we handovered



After you import all sql files, your database should be like this, which include tables: `customer`, `forget_pass_token`, `items`, `item_orders`, `orders` and `users`



5. Copy our website folder (name: `team017-app_fit3048`) into `htdocs` folder, the location of `htdocs` folder should be `C:\xampp\htdocs`

Then open cmd(Command Prompt), copy below commands and paste to cmd

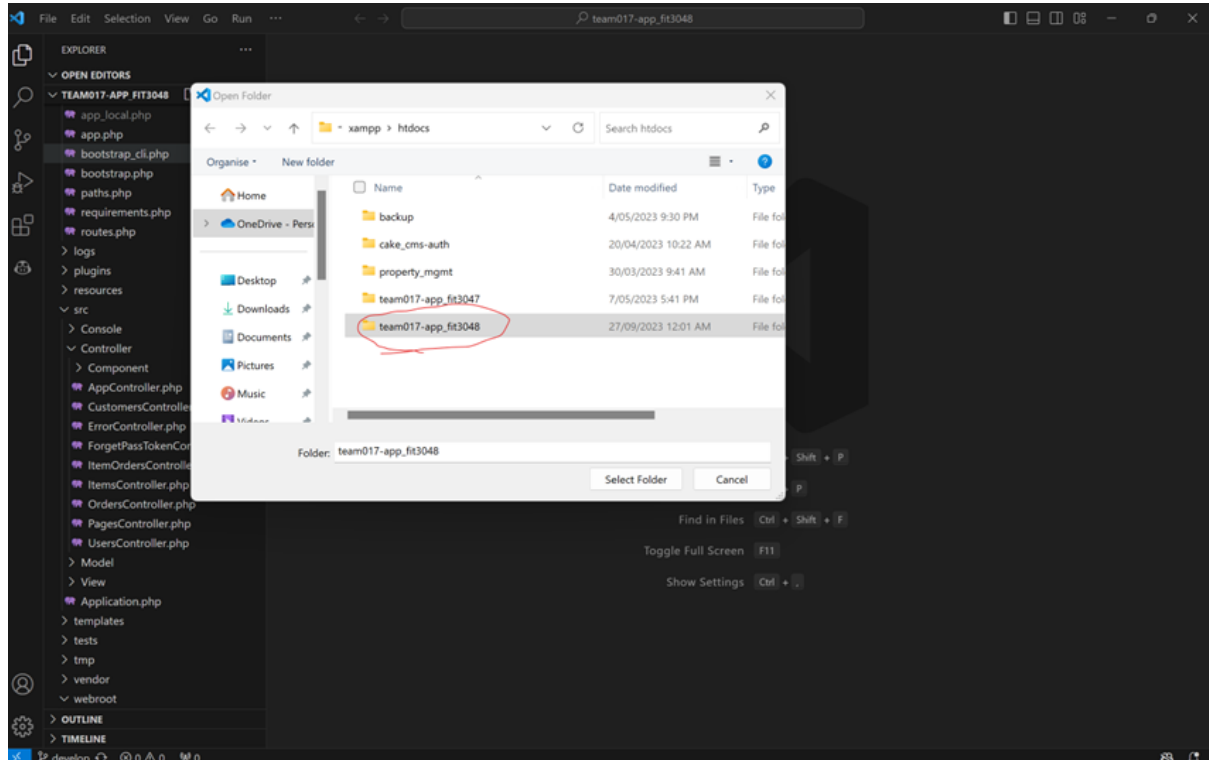
```
cd C:\xampp\htdocs\team017-app_fit3048
```

```
composer install
```

It will install all the project dependencies for you to run that project

6. With the same file path, type in
`composer require "cakephp/authentication:^2.4"`
to install authentication plugin required for login system.

7. Open IDE and choose open folder, then choose the our website folder

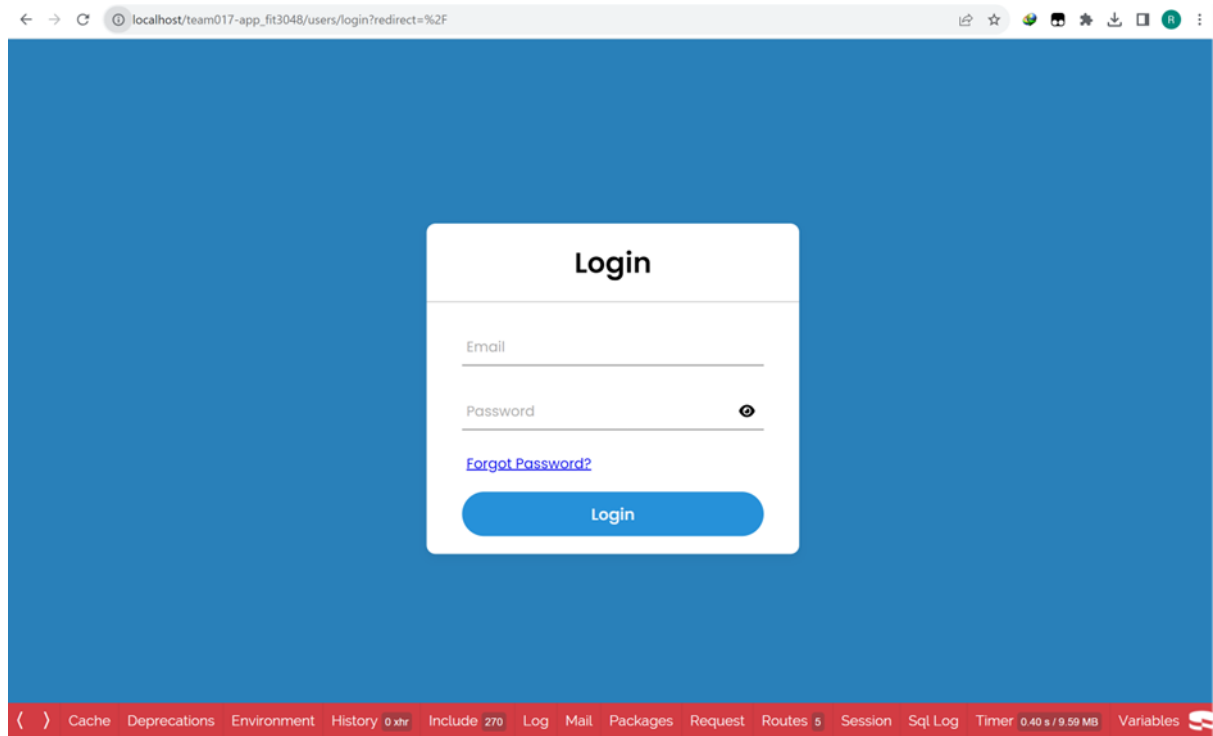


And now you are ready to run and debug the codes

8. If you want to view or test the website locally, type <http://localhost/> in the browser URL and click on our website project



Finally, you will be able to run the website locally on the browser



Required Skills:

- CakePHP (Programming framework)
- Postman (Software)
- REST APIs (Connection to MYOB)
- PHP (Programming language)
- CSS/HTML (Programming language)
- Javascript (Programming language)

External APIs

You will **not** need to sign-up to MYOB. The information you need to connect to MYOB will be given to you

Authorisation:

Endpoint: <https://secure.myob.com/oauth2/v1/authorize/>

Request type: POST

Required headers:

client-id: <client id>

client-secret : <client secret>

grant-type: refresh_token

refresh_token: <refresh token>

Request body: None

Response body:

```
{
  "access_token": <access token>,
  "token_type": "bearer",
  "expires_in": "1200",
  "refresh_token": <new refresh token>,
  "scope": "CompanyFile"
}
```

The new refresh token is used to update the refresh token.
The access token is used as a bearer token to access all the other APIs

All MYOB header requirements:

x-myobapi-key: <client id>
x-myobapi-version: v2
Accept-Encoding: gzip,deflate
Authorization: Bearer <bearer token from authorisation>

These will be all the headers we need to set for the rest of the APIs.

There are some common string formats in the following API responses:
UID String: <8 letters/numbers>-<4 letters/numbers>-<4 letters/numbers>-<4 letters/numbers>-<12 letters/numbers>
URI String: <https://api.myob.com/accountright/><company_file_id>/<some information>/<some more information>/<relevant UID String>/<Extra info if necessary>
Date String:YYYY-MM-DD"T"HH-mm:SS

Load customer API:

Endpoint: <https://api.myob.com/accountright/><company_file_id>/Contact/Customer

Request type: GET

Request body: None

Response body:

```
{
  "Items":
  [
    {
      "UID": UID String,
      "CompanyName": String,
      "IsIndividual": Boolean,
      "DisplayID": String,
    }
  ]
}
```

```

"IsActive": Boolean,
"Addresses": [
  {
    "Location": Number,
    "Street": String,
    "City": String,
    "State": String
    "PostCode": String,
    "Country": String,
    "Phone1": String,
    "Phone2": String,
    "Phone3": String,
    "Fax": String,
    "Email": String,
    "Website": String,
    "ContactName": String,
    "Salutation": String
  }
],
"Notes": String,
"Identifiers": null,
"CustomList1": null,
"CustomList2": null,
"CustomList3": null,
"CustomField1": {
  "Label": String,
  "Value": String
},
"CustomField2": {
  "Label": String,
  "Value": String
},
"CustomField3": {
  "Label": String,
  "Value": String
},
"CurrentBalance": Number,
"SellingDetails": {
  "SaleLayout": String,
  "PrintedForm": String,
  "InvoiceDelivery": String,
  "ItemPriceLevel": String,
  "IncomeAccount": String,
  "ReceiptMemo": String,
  "SalesPerson": String,
  "SaleComment": String,
  "ShippingMethod": String,
  "HourlyBillingRate": Number,

```

```

"ABN": "",
"ABNBranch": "",
"TaxCode": {
  "UID": UID String,
  "Code": String,
  "URI": URI String
},
"FreightTaxCode": {
  "UID": UID String,
  "Code": String,
  "URI": URI String
},
"UseCustomerTaxCode": Boolean,
"Terms": {
  "PaymentIsDue": String,
  "DiscountDate": Number,
  "BalanceDueDate": Number,
  "DiscountForEarlyPayment": Number,
  "MonthlyChargeForLatePayment": Number,
  "VolumeDiscount": Number
},
"Credit": {
  "Limit": Number,
  "Available": Number,
  "PastDue": Number,
  "OnHold": Boolean
},
"TaxIdNumber": String,
"Memo": String
},
"PaymentDetails": String,
"ForeignCurrency": null,
"LastModified": Date String,
"PhotoURI": URI String,
"URI": URI String,
"RowVersion": String
}
],
"NextPageLink": URI String,
"Count": Number
}

```

This returns a list of “Items”, one item being a customer. The following values will be based on a single customer.

Fields we use:
 UID
 CompanyName

DisplayID
Addresses.Street
Addresses.City
Addresses.State
Addresses.Postcode
URI

We also use the “NextPageLink” to loop through all the customers.

Load customer API:

Endpoint: https://api.myob.com/accountright/<company_file_id>/Inventory/Item

Request type: GET

Request body: None

Response body:

```
{
  "Items": [
    {
      "UID": UID
      String,
      "Number": String,
      "Name": String,
      "IsActive": Boolean,
      "Description": String,
      "UseDescription": Boolean,
      "CustomList1": null,
      "CustomList2": {
        "Label": "Wholesale Price",
        "Value": String
      },
      "CustomList3": null,
      "CustomField1": null,
      "CustomField2": {
        "Label": "Barcode",
        "Value": String
      },
      "CustomField3": {
        "Label": "Bale Qty",
        "Value": String
      },
      "QuantityOnHand": Number,
      "QuantityCommitted": Number,
      "QuantityOnOrder": Number,
      "QuantityAvailable": Number,
      "AverageCost": Number,
```

```

"CurrentValue": Number,
"BaseSellingPrice": Number,
"IsBought": Boolean,
"IsSold": Boolean,
"IsInventoried": Boolean,
"ExpenseAccount": String,
"CostOfSalesAccount": {
  "UID": UID,
  String,
  "Name": String,
  "DisplayID": String,
  "URI": URI,
  String
},
"IncomeAccount": {
  "UID": UID,
  String,
  "Name": String,
  "DisplayID": String,
  "URI": URI
  String
},
"AssetAccount": {
  "UID": UID
  String,
  "Name": UID
  String,
  "DisplayID": String,
  "URI": URI
  String
},
"BuyingDetails": {
  "LastPurchasePrice": Number,
  "StandardCost": Number,
  "BuyingUnitOfMeasure": String,
  "ItemsPerBuyingUnit": Number,
  "TaxCode": {
    "UID": UID
    String,
    "Code": String,
    "URI": URI
    String
  },
  "RestockingInformation": {
    "MinimumLevelForRestockingAlert": Number,
    "Supplier": String,
    "SupplierItemNumber": String,
    "DefaultOrderQuantity": Number
  }
}

```

```

    },
    "StandardCostTaxInclusive": Boolean
  },
  "SellingDetails": {
    "BaseSellingPrice": Number,
    "SellingUnitOfMeasure": String,
    "ItemsPerSellingUnit": Number,
    "TaxCode": {
      "UID": UID
      String,
      "Code": String,
      "URI": URI
      String
    },
    "IsTaxInclusive": Boolean,
    "CalculateSalesTaxOn": String,
    "PriceMatrixURI": URI
    String
  },
  "LocationDetails": String,
  "DefaultSellLocation": String,
  "DefaultReceiveLocation": String,
  "LastModified": Date
  String,
  "PhotoURI": URI
  String,
  "URI": URI
  STRING
  "RowVersion": String
}
],
"NextPageLink": URI String
"Count": Number
}

```

This returns a list of “Items”, one item being a single item. The following values will be based on a single item.

Fields we use:

UID
 Number
 DisplayID
 Name
 CustomField1.Value
 CustomField2.Value
 CustomField3.Value
 BaseSellingPrice

QuantityAvailable
CustomList3.Value
PhotoURI
URI

We also use the “NextPageLink” to loop through all the customers.

Create Order API:

Endpoint: https://api.myob.com/accountright/<company_file_id>/Sale/Order/Item

Request type: POST

Request body:

```
{
  "Date": String,
  "ShipToAddress": String,
  "Customer": {
    "UID": UID String
  },
  "Lines": [
    {
      "Type": "Header",
      "Description": "Order for doormats"
    },
    {
      "Type": "Transaction",
      "Description": String,
      "ShipQuantity": String,
      "UnitPrice": Number,
      "DiscountPercent": Number,
      "Total": Number,
      "Item": {
        "UID": UID String
      },
      "TaxCode": {
        "UID": UID String,
        "Code": "GST",
        "URI": URI String
      }
    }
  ],
  "Subtotal": Number,
  "Comment": String
}
```

Response body: None - unless you get a list of errors

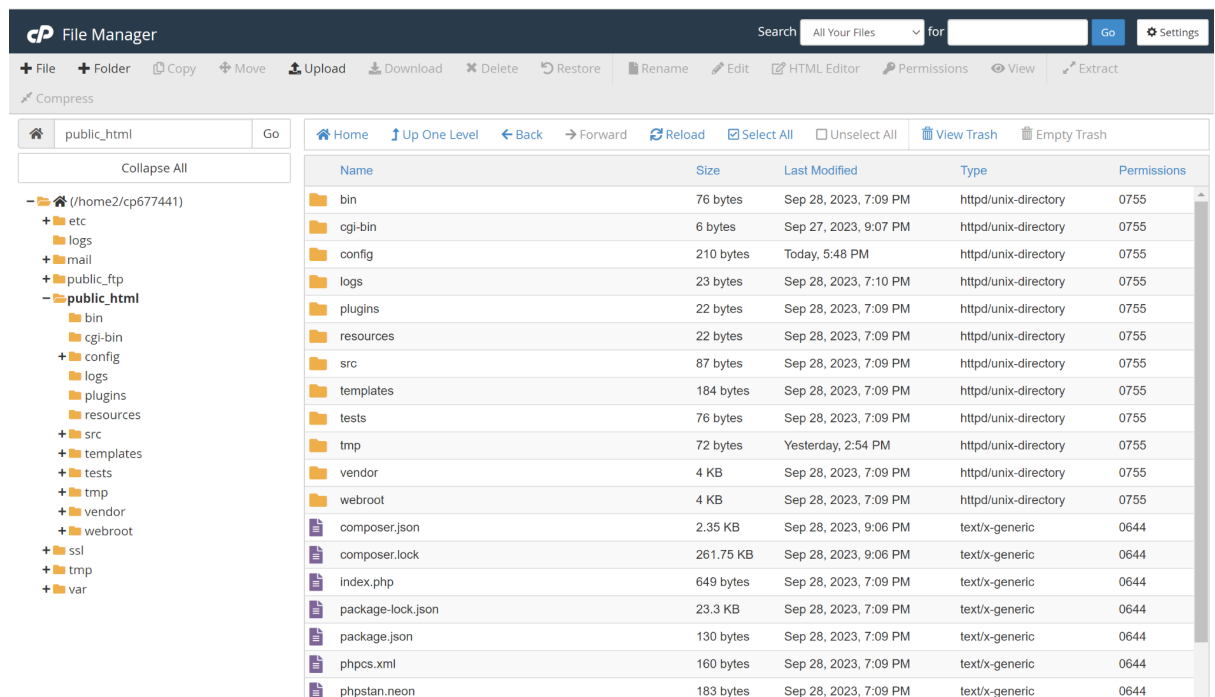
Notes:

- The header line is always the same
- The taxcode will also be the same, but I can't share it here as there is some potentially sensitive information

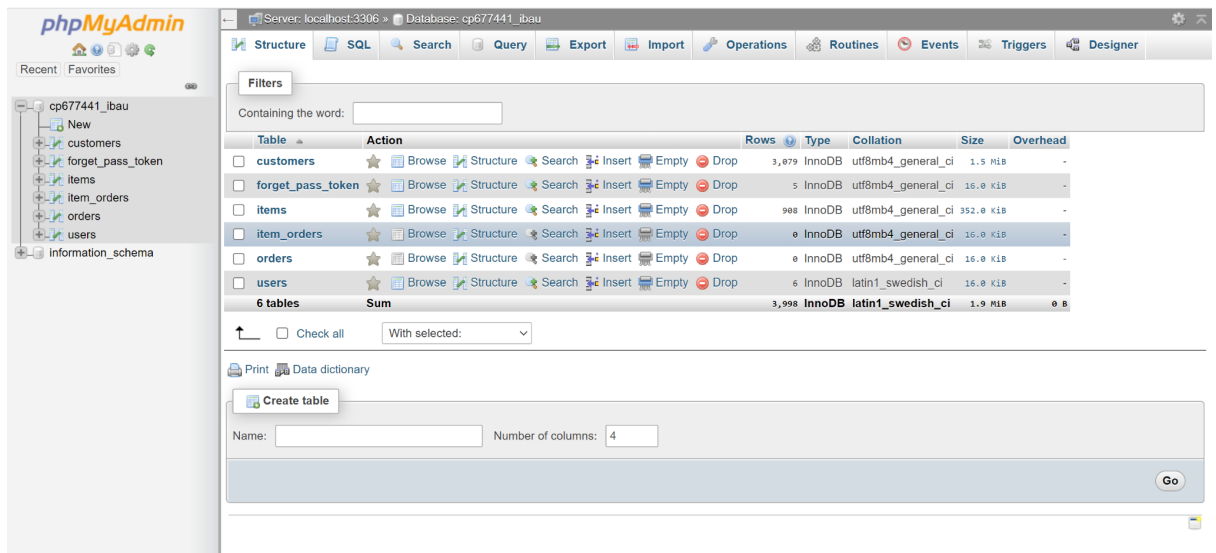
Deployment documentation

Step-by-step process of building and pushing code to the cloud server, as well as initial setups required to make the system fully functional:

1. Login your account on Crazy Domain <https://www.crazydomains.com.au/>
2. Goto Cpanel (Control Panel) and choose File Manager
3. Goto public/html folder and press upload button to upload all the files from the folder we handovered to you on the server
4. Once you uploaded all of the file, your file manager should be like this



5. Then go back to Cpanel, and choose PhpMyadmin
6. Select database named cp677441_ibau, and then import the sql files we handovered
7. The tables are: customer, forget_pass_token, items, item_orders, orders and users. Once you finished import all tables, the database should look like this.



Design Specifications

Typography details

Throughout the system the font used is from the below:

Font Family: 'Font Awesome 5 Free' from Material Dashboard 2

Brand Palette

Colours used throughout the system are variations of the template used. The green matches the logo colour



Each function of the website is separated into colours

- Green is associated with ordering
 - Usage: Search Customer, New Order pages



Name: Success

HEX: #218f53

RGB: 33, 143, 83

Text Colour: White

- Blue is associated with the user management system
 - Used on pages associated with logging in, adding users, viewing users



Name: Info

HEX: #2980b9

RGB: 41, 128, 185

Text colour: White

- Pink is associated with specific actions with more serious and permanent consequences such as adding a new user or deleting a user.
 - Used on pages relating to users, view users + individual users



Name: Primary

HEX: #cc1855

RGB: 204, 24, 85

Text colour: White

UI/UX components

Icons

- Icon packs developers used the Font Awesome icons when building but the Material Dashboard template includes the use of default icons by Google.
- Icon selection for each tile or button is associated with whichever best represented the action
- The font face family used for icons is NucleoIcons and is within the nucleo-icons.css file

<https://fonts.google.com/icons>

<https://fontawesome.com/v5/search>

Logo designs

- Logo designs are consistent with the use of IB Australia. Throughout the system this is with the use of the IB Australia logo but changes to the Solemate Doormat to be sent via email when forgot password is used. The referred to logos are as below.
- These logos are protected by copyright do not recreate.



Images

- All imagery used throughout the system is aimed to replicate the style of doormats sold by IB Australia. While these do not come directly from IB Australia these are still chosen to mimic the same style.
- An example of our selected doormat used on our dashboard



An example of a doormat from IB Australia (Printed PVC Backed Coir)



Bootstrap design templates for the below features & related CSS file in system:

- Login: Simple Login Form with Floating Label Animation
 - o CSS file: login.css
 - URL: <https://www.codingnepalweb.com/free-login-registration-form-html-css/>
- Dashboard, order creation and user management: Material Dashboard 2
 - o CSS file: material-dashboard.css
 - URL: <https://www.creative-tim.com/product/material-dashboard>
- Error pages: Colorlib 404 v20 Error Template
 - o CSS file: error404.css
 - URL: <https://colorlib.com/wp/free-404-error-page-templates/>
- Email Layout: MDB Plain Text Email Template
 - o CSS file: emailLayout.css
 - URL: <https://mdbootstrap.com/docs/b4/jquery/css/e-mail-templates/>